

Why Good Docstrings Are Not Good Enough

Five well-documented Python tools, fifteen ordinary requests, and the nine wrong answers nobody would have noticed.

9 of 15

requests got the
wrong arguments

0

of those nine
showed an error

15 of 15

correct after the
rewrite

Prepared for Sourav Bhattacharya

Created Sunday, 06 September 2026 at 15:28 (PDT)

Method 5 tools, asked 3 different ways each, tested three times

Code delivered with this document, runnable as-is

Contents

1. The short version	3
2. The five tools we tested	4
3. How we tested	5
Why three phrasings?	5
4. The scoreboard	6
5. The nine mistakes, one at a time	7
5.1 The meeting booked at the wrong hour (2 mistakes)	7
5.2 A company name where an ID belongs	8
5.3 A status word that was not on the list	9
5.4 Two currencies swapped	10
5.5 A text message sent to an email address	11
5.6 A priority level that does not exist	12
5.7 A picture size the tool cannot do (2 mistakes)	13
6. Six rules that came out of this	14
7. Doing the checks with Pydantic	15
7.1 What the model looks like	15
7.2 One model, two jobs	16
7.3 Where Pydantic's own message is not enough	17
7.4 Everything wrong at once	18
7.5 Every check, and what it says	19
8. The retest	21
9. What the input checks cannot catch	22
10. A checklist for your next tool	23
11. References	23
Appendix A — the finished tools	25
Appendix B — how to run this yourself	34

1. The short version

We wrote five ordinary Python tools. Each one had a careful docstring: every parameter had a clear name, a stated type, and an example. Then we gave each tool to an AI model and asked for the same thing three different ways, in plain English, and watched which arguments it filled in.

Out of 15 requests, the model got 6 right and 9 wrong.

That is not the surprising part. This is: **not one of the nine wrong calls produced an error.** The tool took the bad argument, did something with it, and returned a normal-looking result. The model then told the user the job was done.

Here is what that looks like in practice:

- A meeting booked for 3pm was actually booked for 8am. The confirmation still said 3pm.
- A search for a customer's orders returned an empty list. It read like “this customer has no orders”, not like “your search was broken”.
- A text message was addressed to an email address and handed to the SMS system. Nobody received it.

After we rewrote the nine descriptions and added input checks — declared as Pydantic models — the same 15 requests all came back correct, and 18 deliberately broken calls were all rejected with a message saying how to fix them. But the two fixes do different jobs, and it is worth knowing which one is doing the work.

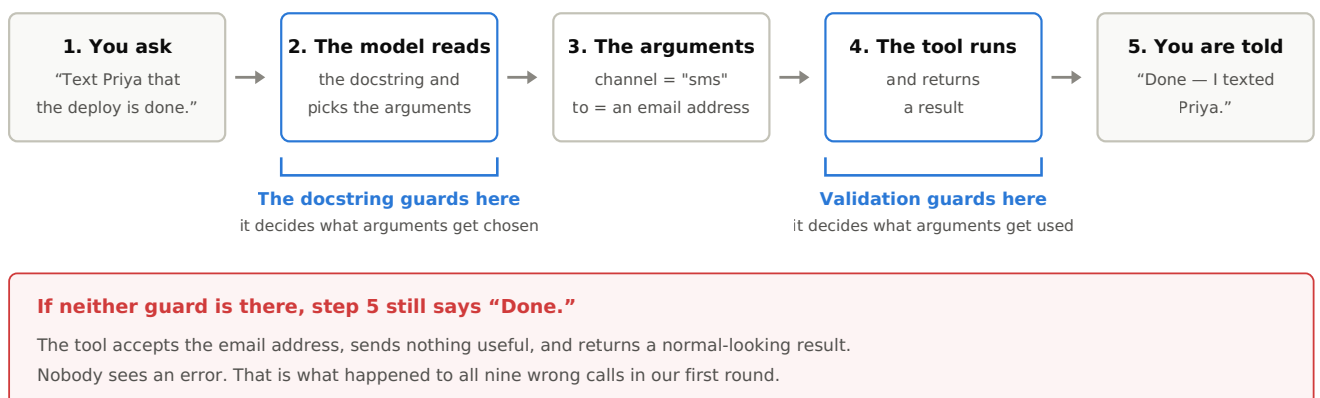


Figure 1. A tool call passes two gates. The docstring decides what arguments get picked. Validation decides what arguments get used. If a value looks fine but means the wrong thing, only the first gate can stop it.

■ right arguments ❌ wrong arguments

Before (original docstrings)



After (rewritten + validation)



15 requests in total: 5 tools, asked 3 different ways each.

Figure 2. The same 15 requests, before and after the rewrite.

2. The five tools we tested

They are the kind of tools any internal assistant might have. Each one was chosen because it makes a different kind of mistake easy to make.

TOOL	WHAT IT DOES	WHAT MAKES IT EASY TO GET WRONG
schedule_meeting	Books a meeting.	A time with no timezone looks completely fine and is completely ambiguous.
search_orders	Searches orders.	It only accepts five exact status words, and users never use those words.
convert_currency	Converts money.	Two parameters hold the same kind of value, so they are easy to swap.
send_notification	Sends one message.	The address format changes depending on the channel you pick.
resize_image	Resizes a picture.	People ask for “half the size”, but the tool only understands pixels.

Here are the five, as we first wrote them:

```
schedule_meeting(attendee_emails, start_datetime_iso, duration_minutes, room_name)

search_orders(customer_id, status, created_after, created_before, max_results)

convert_currency(amount, source_currency, target_currency, rate_date)

send_notification(recipient_handle, channel, message_body, priority)

resize_image(source_path, target_path, width_px, height_px,
              preserve_aspect_ratio, jpeg_quality)
```

Read that list and nothing looks wrong. Every name says what it holds. There is no `data`, no `options`, no `flag`. That is the whole point of this report: good names are not enough on their own.

3. How we tested

For each request, the model was shown two things and nothing else:

1. The tool's name, its parameters, and its docstring.
2. A short block of background facts, the same one every time.

The background facts

Today is Sunday, 6 September 2026. The user is in Los Angeles (UTC−07:00 today).

People: Priya Raman, Marco Silva, Dana Wu — all at `@example.com`.

Customer: Acme Robotics, account CUST-40912.

Before each test we wrote down the arguments a correct answer would need. Then we compared what the model actually sent, field by field, and ran the tool with it.

A request counts as wrong if any single argument is wrong — whether or not the tool complained. That distinction matters, so it has two names:

- **A loud mistake** is one the tool rejects. Annoying, but at least somebody finds out.
- **A quiet mistake** is one the tool accepts. It returns a sensible-looking answer built on the wrong input, and nobody finds out.

All nine mistakes in our first round were quiet ones. That is not bad luck. A plain Python function only rejects the values that happen to crash it, and most wrong values do not crash anything.

Why three phrasings?

Because people do not ask for things the same way twice. For each tool, one phrasing happened to use the same words the tool uses, and one or two did not. Almost every request that passed was the one where the user happened to speak the tool's language. Every request that failed was one where the docstring had to do the translating — and could not.

4. The scoreboard

TOOL	WHAT WE ASKED	RESULT
<code>schedule_meeting</code>	Book a 30 minute sync with Priya and Marco tomorrow at 3pm.	wrong, no error
<code>schedule_meeting</code>	Set up an hour-long meeting with Dana in the Bridgewater room on Sept 9 at 10am.	wrong, no error
<code>schedule_meeting</code>	Book 15 minutes with Dana in Bridgewater at 9am Pacific on the 8th.	correct
<code>search_orders</code>	Show me all of Acme's cancelled orders from last week.	wrong, no error
<code>search_orders</code>	What orders have shipped for CUST-40912 in the past 3 days?	correct
<code>search_orders</code>	List CUST-40912's completed orders this month — just the top 5.	wrong, no error
<code>convert_currency</code>	Convert 500 US dollars to euros.	correct
<code>convert_currency</code>	What's the euro price of an \$89 subscription?	wrong, no error
<code>convert_currency</code>	What was \$1,200 worth in pounds on the first of August?	correct
<code>send_notification</code>	Text Priya that the deploy is done.	wrong, no error
<code>send_notification</code>	Send a high-priority Slack message to @dana about the outage.	correct
<code>send_notification</code>	Email marco.silva@example.com the release notes and mark it urgent.	wrong, no error
<code>resize_image</code>	Resize logo.png to 800 pixels wide.	wrong, no error
<code>resize_image</code>	Shrink hero.jpg by half and save it as hero_small.jpg.	wrong, no error
<code>resize_image</code>	Make banner.png exactly 1200x630 for the OG image, high quality.	correct

Three things stand out.

One parameter never failed once: `duration_minutes`. It was filled in correctly for “30 minute”, “hour-long” and “15 minutes”. Nobody ever sent 1 to mean one hour. The reason is simple: the unit is in the name, right where the value gets typed. The docstring did not have to say a word about it.

Mistakes cluster on values the model cannot work out from the sentence. A timezone that lives in someone's profile. An account ID that lives in another system. The size of a file on disk. When the model cannot work out the answer, it does not stop and ask. It writes down its best guess, neatly formatted.

Listing the allowed values is not the same as saying “these are the only ones”. Both of our list-based parameters had their values written out in the docstring. The model still invented new ones.

5. The nine mistakes, one at a time

Each one is told the same way: what was asked, what the model sent, what happened, why, and the one change that fixed it.

5.1 The meeting booked at the wrong hour (2 mistakes)

WHAT YOU ASKED FOR

“Book a 30 minute sync with Priya and Marco tomorrow at 3pm.”

WHAT THE MODEL SENT

```
start_datetime_iso = "2026-09-07T15:00:00"
```

WHAT HAPPENED

The meeting was booked. The confirmation said 15:00. The actual slot was 8am Pacific — seven hours early.

WHY

The time has no timezone on the end. The server assumes a time with no timezone is UTC. The user meant 3pm in Los Angeles. Nothing in the docstring said what happens when the timezone is missing, so the model had no reason to think it mattered — and the example in the docstring was missing it too.

THE FIX

DOCSTRING BEFORE

```
start_datetime_iso : str
    An ISO 8601 datetime string,
    e.g. "2026-09-07T15:00:00".
```

DOCSTRING AFTER

```
start_datetime_with_utc_offset : str
    An ISO 8601 datetime. It MUST end
    with a timezone offset.
    Good: "2026-09-07T15:00:00-07:00"
    Bad:  "2026-09-07T15:00:00"
```

Two changes. The name now says what has to be there. And there is a **Bad** example showing the exact mistake, not just a good one. **The model copies your example.** If your example is missing something, so is the value you get back.

The same mistake happened again on the next request, phrased differently. It only stopped when the docstring changed.

5.2 A company name where an ID belongs

WHAT YOU ASKED FOR

“Show me all of Acme's cancelled orders from last week.”

WHAT THE MODEL SENT

```
customer_id = "Acme Robotics"
```

WHAT HAPPENED

The search ran and found nothing. To the user it looked like Acme had no cancelled orders.

WHY

`customer_id` needs an account ID like `CUST-40912`. The user said “Acme”. The model had no way to turn a company name into an ID, and the docstring never mentioned that there was a way. Faced with a required box and nothing to put in it, the model wrote down the closest thing it had.

THE FIX

DOCSTRING BEFORE

```
customer_id : str
    The customer's account
    identifier, e.g. "CUST-40912".
```

DOCSTRING AFTER

```
customer_account_id : str
    An account ID: "CUST-" plus five
    digits, e.g. "CUST-40912".
    A company name is NOT an ID. If the
    user says "Acme", call
    lookup_customer_id("Acme") first.
    Do not guess or make one up.
```

Telling the model “don't guess” only works if you also tell it where to go instead. Give it somewhere to go and it goes there. Give it a rule with no alternative and it just guesses differently.

5.3 A status word that was not on the list

WHAT YOU ASKED FOR

"List CUST-40912's completed orders this month."

WHAT THE MODEL SENT

```
status = "completed"
```

WHAT HAPPENED

Zero results. Again, it looked like a real answer.

WHY

The tool only accepts five words: `pending`, `paid`, `shipped`, `delivered`, `cancelled`. The word for "completed" is `delivered`. The docstring listed all five, but never said the list was the whole list — so the model added a sixth word that seemed to fit.

THE FIX

DOCSTRING BEFORE

```
status : str
    Filter to orders in this state.
    One of: pending, paid, shipped,
    delivered, cancelled.
```

DOCSTRING AFTER

```
order_status : str
    ONLY these five words are allowed:
    "pending"    placed, not paid yet
    "paid"       paid, not sent yet
    "shipped"    on its way
    "delivered"  arrived (=
"completed")
    "cancelled"  cancelled or refunded
    Users say different words.
    Translate:
    completed / done -> "delivered"
    refunded         -> "cancelled"
    in transit       -> "shipped"
```

Two things were added. First, **ONLY these five are allowed** — which is different from just listing five. Second, a translation table from the words people actually say to the words the tool accepts. Do that translation in the docstring, so the model does not have to improvise it.

5.4 Two currencies swapped

WHAT YOU ASKED FOR

“What's the euro price of an \$89 subscription?”

WHAT THE MODEL SENT

```
source_currency = "EUR"  
target_currency = "USD"
```

WHAT HAPPENED

It converted 89 euros into dollars instead of 89 dollars into euros. The answer came back as €96.74. The right answer was €81.88.

WHY

The sentence mentions euros first and dollars second, but the number (89) is in dollars. To fill in `source_currency`, the model has to work out a direction from the sentence, and the nearest currency word was the wrong one. The docstring said “the currency the money is currently in”, which is correct and did not help.

THE FIX

DOCSTRING BEFORE

```
source_currency : str  
    The currency the money is  
    currently in.  
target_currency : str  
    The currency to convert into.
```

DOCSTRING AFTER

```
amount_currency_code : str  
    The currency `amount` is written in.  
result_currency_code : str  
    The currency the answer comes back  
    in.
```

Ask: "what unit is the NUMBER in?"
That is `amount_currency_code` – no matter
where the words appear in the sentence.

The old names describe a *direction*, which the reader has to reconstruct. The new names describe a *fact*: one is the currency of the number, the other is the currency of the answer. A sentence written backwards can no longer flip them.

5.5 A text message sent to an email address

WHAT YOU ASKED FOR

"Text Priya that the deploy is done."

WHAT THE MODEL SENT

```
channel = "sms"
recipient_handle = "priya.raman@example.com"
```

WHAT HAPPENED

The tool accepted it and reported success. Priya received nothing.

WHY

This was the whole docstring for that parameter: *"Who to notify. The format depends on channel."* It then never said what any of the formats were. The model only had Priya's email address, so that is what it used. A sentence that points at another parameter without answering the question is worse than no sentence, because it looks like documentation.

THE FIX

DOCSTRING BEFORE

```
recipient_handle : str
    Who to notify. The format
    depends on `channel`.
```

DOCSTRING AFTER

```
recipient_address : str
    The address, in the format that
    `channel` requires. They are NOT
    interchangeable:
        "email" -> priya@example.com
        "sms"   -> +14155550142
        "slack" -> @priya
        "push"  -> a 32+ char device token
    You usually only have someone's
    email.
    That does not work on other
    channels:
        call lookup_contact(person,
        channel).
```

If a format depends on another parameter, write out every case. One line each, with a real example. It costs four lines and removes the guesswork completely.

5.6 A priority level that does not exist

WHAT YOU ASKED FOR

"Email marco.silva@example.com the release notes and mark it urgent."

WHAT THE MODEL SENT

```
priority = "urgent"
```

WHAT HAPPENED

Accepted, and passed straight through. Whether it actually counted as urgent depended on what the delivery system did with an unknown word.

WHY

The allowed values are `low`, `normal` and `high`. The user said "urgent", and the model kept the user's word rather than translating it. That is a reasonable instinct. Nothing told it the list was closed, and nothing told it where "urgent" should go.

THE FIX

DOCSTRING BEFORE

```
priority : str
    Delivery priority. One of:
    low, normal, high.
```

DOCSTRING AFTER

```
priority : str
    ONLY these three are allowed:
    "low", "normal", "high".
    Translate what the user says:
    urgent / ASAP / critical -> "high"
    medium / standard       ->
    "normal"
    FYI / no rush           -> "low"
    There is nothing above "high".
```

Same pattern as the status list. Mark the set as closed, then hand over the translation instead of hoping the model guesses it.

5.7 A picture size the tool cannot do (2 mistakes)

WHAT YOU ASKED FOR

“Shrink hero.jpg by half and save it as hero_small.jpg.”

WHAT THE MODEL SENT

```
width_px = 50
height_px = 50
```

WHAT HAPPENED

It produced a 50×50 pixel thumbnail instead of an image half the original size.

WHY

The tool only understands exact pixels, and the model does not know how big the picture currently is. It sent 50 meaning “50 percent”. The tool read it as “50 pixels”. The docstring never said the numbers are absolute pixels, and never said what to do when someone asks for something relative.

THE FIX

DOCSTRING BEFORE

```
width_px : int
    Desired width in pixels.
height_px : int
    Desired height in pixels.
```

DOCSTRING AFTER

This tool understands EXACT PIXELS ONLY. It cannot scale by a percentage, and it does not know the picture's current size.

For "half the size" or "25% smaller": call `get_image_info(source_path)` first, do the maths, then pass real pixels.

Never send 50 to mean "50%".
Never send 0.5 to mean "half".

The most useful thing this docstring now does is say what the tool *cannot* do. On the retest the model called `get_image_info`, read 1920×1080, and sent 960×540. The docstring turned a guess into a lookup.

The second mistake with this tool was smaller but the same shape. Asked to “resize logo.png to 800 pixels wide”, the model sent `width_px=800` and `height_px=800`, because `height_px` was a required parameter and it had to put something there. The fix was to make it optional, and to say so:

```
height_px : int or None
    The height in pixels, or None to work it out from the width.
    "800 pixels wide" -> width_px=800, height_px=None
    Do NOT repeat the width as the height. width_px=800 with
    height_px=800 means "fit inside an 800x800 box", which is a
    different request.
```

If a parameter is required but the caller has no way to know its value, that is a problem with the tool, not with the caller.

6. Six rules that came out of this

Every one of these came from a mistake above, not from taste.

Rule 1

Put the unit or the format in the parameter name.

`duration_minutes` never failed once, because the unit is sitting right there when the value gets written. The same idea fixed the `datetime ( start_datetime_with_utc_offset )` and the currency swap (`amount_currency_code`). If a name needs the docstring to be clear, rename it before you write another word.

Rule 2

Show a bad example, not just a good one.

The model copies your example. Our `datetime` example had no timezone, so neither did the value we got back. Adding one line — *Bad: "2026-09-07T15:00:00" (no offset)* — fixed both timezone mistakes.

Rule 3

Say “these are the only allowed values”, and translate the words people actually use.

Listing five values is not the same as ruling out a sixth. Mark the list closed, then add the mapping: *completed* → *delivered*, *urgent* → *high*. You know that translation. Write it down instead of hoping it gets guessed.

Rule 4

If a format depends on another parameter, list every case.

“The format depends on `channel` ” tells the reader nothing. Four lines, one per channel, each with a real example, tells them everything.

Rule 5

If the caller cannot know a value, say where to get it — then say “do not guess”.

A required box with no way to fill it does not produce a question. It produces a confident invention. Naming the lookup tool is what turns the guess into a lookup; the ban on guessing on its own just produces a different guess.

Rule 6

Say what the tool cannot do.

Docstrings describe what a tool can do and go quiet about the rest. Ask for something outside that, and the request gets squeezed into whatever shape is available. Writing down the limit prevents it.

The rule that did not come out of the data

“Write more detail.” Our first docstrings were already detailed. Every failing parameter had a type and most had an example. What was missing was never volume — it was one specific sentence closing one specific hole. And in three of the nine cases the real fix was **renaming the parameter**, which no amount of extra prose replaces.

7. Doing the checks with Pydantic

A better docstring makes a wrong call less likely. It does not make one impossible, and it does nothing at all when a different model, an older model, or a person calls your tool. So every tool also checks its inputs before doing any work.

You can hand-write those checks with `if` statements. We are using **Pydantic** instead, because for a tool an AI model will call it does something extra that hand-written checks cannot: the same declaration that enforces a rule at runtime also *tells the model the rule up front*.

7.1 What the model looks like

Here is the whole of `search_orders`, checks included:

```
from datetime import date
from typing import Literal, Optional
from pydantic import BaseModel, Field, field_validator, model_validator

SYNONYMS = {"completed": "delivered", "done": "delivered",
            "refunded": "cancelled", "in transit": "shipped"}

class SearchOrdersArgs(BaseModel):

    customer_account_id: Optional[str] = Field(
        None, pattern=r"^CUST-\d{5}$",
        description="An account id: 'CUST-' plus five digits, e.g. "
            "'CUST-40912'. A company name is NOT an id – call "
            "'lookup_customer_id' first. Do not fabricate one.")

    order_status: Literal["pending", "paid", "shipped",
                          "delivered", "cancelled"] = Field(
        description="Users rarely use these words, so translate: "
            "completed/done -> delivered; refunded -> cancelled.")

    created_on_or_after: date
    created_strictly_before: date
    max_results: int = Field(20, ge=1, le=500)
```

```

@model_validator(mode="after")
def window_must_contain_a_day(self):
    if self.created_strictly_before <= self.created_on_or_after:
        raise ValueError(
            "the window covers no days. The upper bound is "
            "exclusive: to include today, pass tomorrow's date.")
    return self

```

Reading that top to bottom:

- `Literal[...]` is the closed list. There is no way to pass a sixth value.
- `pattern=` is the account-id format, written as a regular expression.
- `ge=1, le=500` is the allowed range for `max_results`.
- `description=` is the sentence a human — or a model — reads.
- `@model_validator` is for rules involving more than one field, like the date window.

Every rule that used to be split between the docstring and a block of `if` statements now lives in one place.

7.2 One model, two jobs

This is the part worth understanding. A Pydantic model is not only a validator. It also generates a JSON Schema, and that schema is exactly what tool-calling frameworks send to the model as the tool's description.

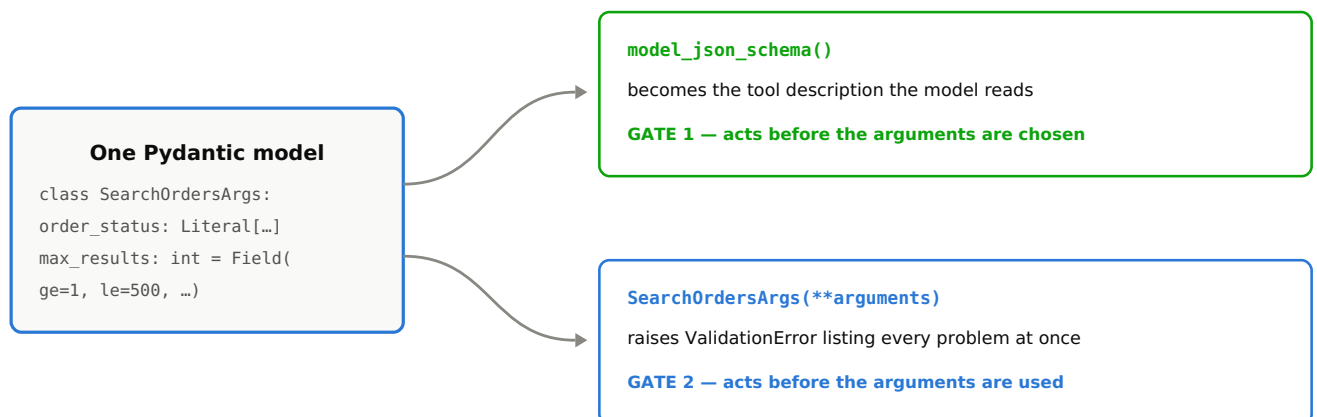


Figure 3. The same declaration does both jobs. Compare it with Figure 1: the top arrow is the first gate and the bottom arrow is the second. Written once, not twice.

Here is part of what `SearchOrdersArgs.model_json_schema()` actually returns:

```

{
  "order_status": {
    "type": "string",
    "enum": ["pending", "paid", "shipped", "delivered", "cancelled"],
    "description": "Users rarely use these words, so translate: ..."
  },
  "max_results": {
    "type": "integer",
    "minimum": 1,

```

```
"maximum": 500,
"default": 20
}
```

The `enum` list is now in the tool definition itself. The model sees the five allowed words *before* it picks one, instead of finding out afterwards that it picked wrong. That is a real change, not a tidiness argument: it moves the constraint from the second gate to the first, which is where this report keeps saying the important work happens.

It also closes a maintenance trap. In the hand-written version the five status words appeared twice — once in the docstring, once in the check — and nothing kept them in step. Here they are written once.

7.3 Where Pydantic's own message is not enough

Pydantic writes its own error messages, and for simple rules they are fine. Send an empty `message_body` and you get:

```
message_body: String should have at least 1 character
```

Clear enough. But for the mistakes in Section 5 the default is not what you want. Pydantic will tell the model that `"completed"` is not in the list. It will not tell it that the word it wants is `"delivered"`, and that suggestion is the whole point.

So near misses get a custom validator:

```
@field_validator("order_status", mode="before")
@classmethod
def suggest_the_right_word(cls, v):
    word = str(v).strip().lower()
    if word in SYNONYMS:
        raise ValueError(
            f"'{v}' is not one of the accepted values. "
            f"Allowed values: pending, paid, shipped, delivered, "
            f"cancelled. '{word}' most likely means "
            f"'{SYNONYMS[word]}' – pass that instead.")
    return word
```

Which produces this:

```
order_status: 'completed' is not one of the accepted values. Allowed values: pending, paid,
shipped, delivered, cancelled. 'completed' most likely means 'delivered' – pass that instead.
```

That gives you the division of labour: **let Pydantic declare the shape, and write a custom validator only where the wording matters**. Across our five tools that came to eleven custom validators out of roughly forty rules. The other twenty-nine are one-liners inside a `Field(...)`.

The message says which argument is wrong	<code>order_status:</code>	So the model knows what to change.
It repeats what was actually sent	<code>Received 'completed'.</code>	So the model can see its own mistake.
It lists the allowed values and the exact fix	<code>Allowed values: pending, paid, shipped, delivered, cancelled. 'completed' most likely means 'delivered' - pass that instead.</code>	So the model knows what to send next time.

Figure 4. Whether a message comes from Pydantic or from your own validator, it wants the same three parts. The third is the one usually missing: a caller told what is wrong, but not what to do instead, tends to retry with the same kind of mistake.

7.4 Everything wrong at once

Hand-written checks stop at the first problem. Pydantic collects all of them and raises one error. Here is a single call with three bad arguments:

```
send_notification(recipient_address="priya.raman@example.com",
                  channel="text",
                  message_body="",
                  priority="urgent")
```

```
channel: 'text' is not one of the accepted values. Allowed values: email, sms, slack, push.
'text' most likely means 'sms' - pass that instead.
message_body: String should have at least 1 character
priority: 'urgent' is not one of the accepted values. Allowed values: low, normal, high.
'urgent' most likely means 'high' - pass that instead.
```

Three problems, one round trip. With stop-at-the-first checks that is three retries, and after each one the model has to guess whether it is finished.

One practical note. Pydantic's own `str(error)` also carries the error type, the input value and a documentation URL — useful to a developer reading a stack trace, noise to a model deciding what to send next. The messages above went through a six-line helper that keeps one line per bad field:

```
def explain(exc: ValidationError) -> str:
    """One line per bad field: the field name and the fix."""
    lines = []
    for err in exc.errors():
        field = ".".join(str(p) for p in err["loc"]) or "(whole call)"
        msg = err["msg"].removeprefix("Value error, ")
        lines.append(f"{field}: {msg}")
    return "\n".join(lines)
```

And one thing comes free: Pydantic's `ValidationError` is a subclass of `ValueError`, so anything already written to catch `ValueError` keeps working unchanged.

7.5 Every check, and what it says

The same 18 deliberately broken calls, now run against the Pydantic version. All 18 are rejected, and all 18 are catchable as `ValueError`.

TOOL	THE BROKEN CALL	RESULT
<code>schedule_meeting</code>	one email address instead of a list of them	caught
<code>schedule_meeting</code>	people's names instead of email addresses	caught
<code>schedule_meeting</code>	a time with no timezone on the end	caught
<code>schedule_meeting</code>	1 sent to mean "one hour"	caught
<code>search_orders</code>	a company name where an account ID belongs	caught
<code>search_orders</code>	the made-up status "completed"	caught
<code>search_orders</code>	"last week" instead of a real date	caught
<code>search_orders</code>	a date range that covers no days at all	caught
<code>convert_currency</code>	the text "1,200" instead of the number 1200	caught
<code>convert_currency</code>	"dollars" instead of "USD"	caught
<code>convert_currency</code>	the same currency on both sides	caught
<code>send_notification</code>	an email address sent by SMS	caught
<code>send_notification</code>	the priority "urgent", which is not on the list	caught
<code>send_notification</code>	the channel "text" instead of "sms"	caught
<code>resize_image</code>	0.5 sent to mean "half the size"	caught
<code>resize_image</code>	no width and no height given	caught
<code>resize_image</code>	an exact size with only one side given	caught
<code>resize_image</code>	the text "800px" instead of the number 800	caught

Four of the messages, exactly as they come out

1 sent to mean "one hour"

```
duration_minutes: 1 is below the 5 minute minimum. This field is in MINUTES, not hours – if the user asked for 1 hour(s), pass 60.
```

a company name where an account ID belongs

```
customer_account_id: 'Acme Robotics' is not a valid account id (expected 'CUST-' followed by five digits, e.g. 'CUST-40912'). If this is a company or person name, resolve it with lookup_customer_id first, or pass null to search all customers. Do not fabricate an id.
```

an email address sent by SMS

(whole call): recipient_address 'priya.raman@example.com' is not valid for channel='sms', which requires an E.164 phone number starting with '+'. Example: '+14155550142'. You passed an email address with channel='sms'. Call lookup_contact(person, 'sms') to get their sms address, or send it by email.

0.5 sent to mean “half the size”

width_px: 0.5 looks like a scale factor, not a pixel count. This tool takes absolute pixels only – call get_image_info(source_path), multiply the real dimension yourself, and pass the result, e.g. 960.

height_px: 0.5 looks like a scale factor, not a pixel count. This tool takes absolute pixels only – call get_image_info(source_path), multiply the real dimension yourself, and pass the result, e.g. 960.

8. The retest

Same 15 requests. Same background facts. New descriptions. **All 15 correct.** This was run twice — once against the hand-written checks and once against the Pydantic models — with identical results, which is the point: the rewrite is what fixed the calls, and the checks are there for when it does not.

TOOL	WHAT WE ASKED	BEFORE	AFTER
<code>schedule_meeting</code>	Book a 30 minute sync with Priya and Marco tomorrow at 3pm.	wrong	correct
<code>schedule_meeting</code>	Set up an hour-long meeting with Dana in the Bridgewater room on Sept 9 at 10am.	wrong	correct
<code>schedule_meeting</code>	Book 15 minutes with Dana in Bridgewater at 9am Pacific on the 8th.	correct	correct
<code>search_orders</code>	Show me all of Acme's cancelled orders from last week.	wrong	correct
<code>search_orders</code>	What orders have shipped for CUST-40912 in the past 3 days?	correct	correct
<code>search_orders</code>	List CUST-40912's completed orders this month — just the top 5.	wrong	correct
<code>convert_currency</code>	Convert 500 US dollars to euros.	correct	correct
<code>convert_currency</code>	What's the euro price of an \$89 subscription?	wrong	correct
<code>convert_currency</code>	What was \$1,200 worth in pounds on the first of August?	correct	correct
<code>send_notification</code>	Text Priya that the deploy is done.	wrong	correct
<code>send_notification</code>	Send a high-priority Slack message to @dana about the outage.	correct	correct
<code>send_notification</code>	Email marco.silva@example.com the release notes and mark it urgent.	wrong	correct
<code>resize_image</code>	Resize logo.png to 800 pixels wide.	wrong	correct
<code>resize_image</code>	Shrink hero.jpg by half and save it as hero_small.jpg.	wrong	correct
<code>resize_image</code>	Make banner.png exactly 1200x630 for the OG image, high quality.	correct	correct

Two of the fixes changed what the model *did*, not just what it typed.

- For “shrink hero.jpg by half”, it stopped guessing and called `get_image_info` first.
- For “Acme's orders”, it stopped inventing an ID and called `lookup_customer_id`.

Both worked for the same reason: the docstring gave the model somewhere else to go.

9. What the input checks cannot catch

We then asked a separate question. If a model still makes one of the original nine mistakes, do the new checks stop it? For six of them, yes. For three, no.

Validation catches these	6 of 9	Only the docstring catches these	3 of 9
The value is the wrong shape, so a check can spot it.		The value looks perfectly fine. It just means the wrong thing.	
- Wrong timezone ×2 - Company name, not an ID - A status word that isn't allowed - Email address on the SMS channel - Priority "urgent"		- Two currencies swapped - Height copied from the width 50 meaning "50 percent"	

Figure 5. The three on the right are the ones to worry about. Every one of them is a perfectly valid value that simply means the wrong thing.

Look at why those three get through:

- `height_px = 800` is a perfectly good pixel count. It is just not the one the user asked for.
- `width_px = 50` is a perfectly good pixel count. The model meant 50 percent.
- `amount_currency_code = "EUR"` is a real, supported currency code. It is just the wrong side of the conversion.

No check can tell those apart from correct calls, because the thing that would tell them apart — what the user actually meant — is not in the arguments at all.

Pydantic does not change this. It is worth being clear about, because declaring a field as `Literal["USD", "EUR", "GBP", "JPY", "INR"]` looks like it should help. It does not: both "USD" and "EUR" are legal members of that list, so a swap sails through. The same goes for `Field(ge=1, le=20000)` on a pixel count — 800 and 50 both pass. No validation library can do better, because the thing that separates a right value from a wrong one is not a property of the value.

That leaves exactly one place to catch them: the description text, read before the arguments are chosen. This is the opposite of the usual instinct, where runtime checks are the real defence and documentation is what you write afterwards if there is time.

The one-line version

Input checks catch calls that are **malformed**. Only the docstring catches calls that are **wrong**. You need both, and one cannot replace the other.

10. A checklist for your next tool

Go through this once per parameter.

1. **Does the name say the unit, the format, or the frame of reference?** If it needs the docstring to make sense, rename it first.
2. **Is your example complete?** Whatever your example is missing, the value you get back will be missing too.
3. **Did you show a bad example as well as a good one?**
4. **If there is a fixed list of values, did you say it is the only list, and did you translate everyday words onto it?**
5. **If the format depends on another parameter, did you write out every case?**
6. **Can the caller actually find this value?** If not, name the tool that provides it and say “do not guess”.
7. **Did you say what the tool cannot do?**
8. **If a date range has an edge, did you say whether it is included, and show the awkward case?**
9. **Have you declared what you can declare?** A closed list is a `Literal`, a format is a `pattern`, a range is `ge/le`. Declared rules end up in the schema the model reads; hand-written ones do not.
10. **Does a bad value raise an error that names the parameter, repeats what arrived, and states the fix?** Pydantic gives you the first two. The third usually needs a custom validator.
11. **Did you test it with three phrasings you did not write while writing the docstring?**
The one that passes is usually the one where you accidentally used your own tool's words.
The other two are the real test.

11. References

1. The code and results from this study, delivered alongside this document: `tools_v1.py` (the original tools), `tools_v2.py` (the fixed tools), `cases.py` and `cases_v2.py` (the 15 requests and the arguments the model sent), `run_round1.py` and `run_round2.py` (the test runners), and the raw results as JSON.
2. ISO 8601 — the international standard for writing dates and times. This is where the timezone offset requirement comes from.
3. ISO 4217 — the standard three-letter currency codes (USD, EUR, GBP).
4. ITU-T Recommendation E.164 — the international phone number format used for the SMS channel (a +, then country code, then digits).
5. PEP 257, *Docstring Conventions*, and the NumPy docstring style — the docstring layout used throughout.

6. Python documentation, *Built-in Exceptions* — `ValueError` is defined as the error for a value of the right type but the wrong content, which is exactly what every check here is testing.
7. Pydantic v2 documentation — *Fields* (constraints and description), *Validators* (`field_validator` and `model_validator`), and *JSON Schema* (`model_json_schema()`). Pydantic 2.13 was used here.
8. JSON Schema, *Draft 2020-12* — the schema format Pydantic emits and that tool-calling APIs accept as a tool's input specification.

Appendix A — the finished tools

This is `tools_v3.py`: the five tools with the rewritten descriptions and every check declared in a Pydantic model. Two other versions ship alongside it — `tools_v1.py` is the original that failed nine times, and `tools_v2.py` is the same fix written with hand-rolled `if` statements, if you want to compare the two styles.

```
"""
tools_v3.py — the five tools, with Pydantic doing the validation.

Same five tools as tools_v2.py and the same rewritten docstrings. The
difference is that every check now lives in a Pydantic model instead of
hand-written `if` statements.

Why that is an improvement for an LLM-facing tool:

1. The constraints become part of the tool DESCRIPTION, not just the
runtime check. `model_json_schema()` emits the enums, patterns, ranges
and per-field descriptions, and that schema is what gets sent to the
model. A `Literal[...]` is therefore visible to the model BEFORE it
chooses a value, instead of only being enforced after.
2. Every problem is reported at once. A model fixing four bad arguments
needs one retry, not four.
3. There is one source of truth. In the hand-written version the allowed
values appeared twice — in the docstring and in the check — and could
drift apart.

What Pydantic does NOT change: a value that is well-formed but means the
wrong thing still passes. See `convert_currency` below.
"""

import re
from datetime import date, datetime
from typing import Annotated, Literal, Optional

from pydantic import (AwareDatetime, BaseModel, ConfigDict, Field,
                      ValidationError, field_validator, model_validator)

# -----
# Shared vocabulary
# -----
OrderStatus = Literal["pending", "paid", "shipped", "delivered", "cancelled"]
Priority = Literal["low", "normal", "high"]
Channel = Literal["email", "sms", "slack", "push"]
CurrencyCode = Literal["USD", "EUR", "GBP", "JPY", "INR"]

STATUS_SYNONYMS = {"completed": "delivered", "complete": "delivered",
                  "fulfilled": "delivered", "done": "delivered",
                  "refunded": "cancelled", "void": "cancelled",
                  "voided": "cancelled", "in transit": "shipped",
                  "dispatched": "shipped", "unpaid": "pending",
                  "awaiting payment": "pending"}
PRIORITY_SYNONYMS = {"urgent": "high", "asap": "high", "critical": "high",
                    "p0": "high", "emergency": "high", "important": "high",
                    "medium": "normal", "standard": "normal",
                    "fyi": "low", "whenever": "low", "no rush": "low"}
CHANNEL_SYNONYMS = {"text": "sms", "txt": "sms", "message": "slack",
                   "dm": "slack", "mail": "email", "e-mail": "email"}
CURRENCY_NAMES = {"DOLLAR": "USD", "DOLLARS": "USD", "$": "USD", "USD ": "USD",
                  "EURO": "EUR", "EUROS": "EUR", "€": "EUR",
                  "POUND": "GBP", "POUNDS": "GBP", "£": "GBP",
                  "YEN": "JPY", "¥": "JPY",
                  "RUPEE": "INR", "RUPEES": "INR", "₹": "INR"}

RE_EMAIL = re.compile(r"^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$")
```

```

RE_E164 = re.compile(r"^[+][1-9]\d{7,14}$")
RE_SLACK = re.compile(r"^(?@[a-z0-9._-]+|[UW][A-Z0-9]{6,})$")
RE_PUSH = re.compile(r"^[A-Za-z0-9_-]{32,}$")
RE_OFFSET = re.compile(r"(Z|[-+]\d{2}:?\d{2})$")

FX = {"USD": 1.0, "EUR": 0.92, "GBP": 0.79, "JPY": 156.4, "INR": 83.2}

EmailAddress = Annotated[str, Field(pattern=r"^[^@\\s]+@[^@\\s]+\\.([^@\\s]+)$")]

def _closed_set_message(param, value, allowed, synonyms):
    """The message shape used for every closed vocabulary."""
    key = str(value).strip().lower()
    allowed_txt = ", ".join(allowed)
    if key in synonyms:
        return (f"'{value}' is not one of the accepted values. "
                f"Allowed values: {allowed_txt}. "
                f"'{key}' most likely means '{synonyms[key]}' – pass that instead.")
    return (f"'{value}' is not one of the accepted values. "
            f"Allowed values: {allowed_txt}. This set is closed; do not invent "
            f"a synonym. If none of these fit, tell the user rather than guessing.")

def explain(exc: ValidationError) -> str:
    """Turn a ValidationError into the shortest thing worth showing a model.

    Pydantic's own string carries the error type, the input value and a
    documentation URL, which is useful to a developer and noise to a model.
    This keeps one line per bad field: the field name and the fix.
    """
    lines = []
    for err in exc.errors():
        field = ".".join(str(p) for p in err["loc"]) or "(whole call)"
        msg = err["msg"].removeprefix("Value error, ")
        lines.append(f"{field}: {msg}")
    return "\n".join(lines)

class StrictModel(BaseModel):
    """Base for every tool's arguments: reject unknown fields outright."""
    model_config = ConfigDict(extra="forbid", str_strip_whitespace=True)

# -----
# Tool 1 – schedule_meeting
# -----
class ScheduleMeetingArgs(StrictModel):
    """Arguments for schedule_meeting."""

    attendee_emails: list[EmailAddress] = Field(
        min_length=1,
        description=(
            "A LIST of full email addresses, one per invitee, even when there "
            "is only one. Each item must match name@domain.tld. "
            "Correct: ['priya.raman@example.com']. "
            "Rejected: 'priya.raman@example.com' (a string, not a list). "
            "Rejected: ['Priya'] (a display name, not an address) – resolve "
            "names with lookup_person first; do not guess an address.")
    )

    start_datetime_with_utc_offset: AwareDatetime = Field(
        description=(
            "The instant the meeting starts, as an ISO 8601 datetime that MUST "
            "carry a UTC offset or a trailing 'Z'. "
            "Good: '2026-09-07T15:00:00-07:00' (3pm in Los Angeles). "
            "Good: '2026-09-07T22:00:00Z' (the same instant, in UTC). "
            "Bad: '2026-09-07T15:00:00' (no offset – ambiguous). "
            "When the user says a wall-clock time such as '3pm', that is a "
            "wall-clock time in THEIR timezone: append their current UTC "
            "offset. Do not relabel local time as UTC.")
    )

    duration_minutes: int = Field(

```

```

ge=5, le=480,
description="Meeting length in whole MINUTES, 5 to 480. "
            "'half an hour' -> 30, 'an hour' -> 60.")

room_name: Optional[str] = Field(
    None, min_length=1,
    description="Exact name of a physical room, e.g. 'Bridgewater'. Use "
                "null for a video-only meeting – not '' and not 'none'.")

@field_validator("attendee_emails", mode="before")
@classmethod
def _must_be_a_list(cls, v):
    if isinstance(v, str):
        raise ValueError(
            "expected a list of email addresses, got a single string. "
            "Wrap it in a list, e.g. ['priya.raman@example.com'].")
    if isinstance(v, (list, tuple)):
        names = [x for x in v
                  if isinstance(x, str) and not RE_EMAIL.match(x.strip())]
    if names:
        raise ValueError(
            f"{names} are not email addresses. Every item must be a "
            f"full address (name@domain.tld). These look like display "
            f"names – resolve them with lookup_person first; do not "
            f"guess an address.")

    return v

@field_validator("duration_minutes", mode="before")
@classmethod
def _minutes_not_hours(cls, v):
    if isinstance(v, int) and not isinstance(v, bool) and 1 <= v <= 4:
        raise ValueError(
            f"{v} is below the 5 minute minimum. This field is in MINUTES, "
            f"not hours – if the user asked for {v} hour(s), pass "
            f"{v * 60}.")

    return v

@field_validator("start_datetime_with_utc_offset", mode="before")
@classmethod
def _must_carry_an_offset(cls, v):
    if isinstance(v, str) and not RE_OFFSET.search(v.strip()):
        raise ValueError(
            "has no UTC offset, so the intended instant is ambiguous. "
            "Append the user's current UTC offset (or 'Z' for UTC), e.g. "
            "'2026-09-07T15:00:00-07:00' for 3pm Pacific. There is no "
            "default timezone.")

    return v

@field_validator("room_name", mode="before")
@classmethod
def _no_fake_rooms(cls, v):
    if isinstance(v, str) and v.strip().lower() in {"", "none", "null", "n/a"}:
        raise ValueError(
            "expected a real room name. Use null (not '' or 'none') for a "
            "video-only meeting.")

    return v

def schedule_meeting(**kwargs):
    """Book a meeting on the shared team calendar.

    See ScheduleMeetingArgs for every parameter, its type and its allowed
    values. That model is also what generates the tool's JSON Schema, so the
    rules below are visible to the caller before it picks any values.

    Raises
    -----
    pydantic.ValidationError
        A subclass of ValueError. Lists every bad argument at once.
    """
    a = ScheduleMeetingArgs(**kwargs)

```

```

end = datetime.fromtimestamp(
    a.start_datetime_with_utc_offset.timestamp() + a.duration_minutes * 60,
    a.start_datetime_with_utc_offset.tzinfo)
return {"event_id": "evt_0001",
        "starts_at": a.start_datetime_with_utc_offset.isoformat(),
        "ends_at": end.isoformat(),
        "invited": a.attendee_emails, "room": a.room_name}

# -----
# Tool 2 – search_orders
# -----
class SearchOrdersArgs(StrictModel):
    """Arguments for search_orders."""

    customer_account_id: Optional[str] = Field(
        None, pattern=r"^CUST-\d{5}$",
        description=(
            "An account id: the literal text 'CUST-' followed by exactly five "
            "digits, e.g. 'CUST-40912'. A company name is NOT an account id – "
            "if the user says 'Acme', call lookup_customer_id('Acme') first. "
            "Do not guess or fabricate an id. Use null to search all customers."))

    order_status: OrderStatus = Field(
        description=(
            "CLOSED SET – exactly one of: pending (placed, not yet paid), paid "
            "(captured, not yet dispatched), shipped (with the carrier), "
            "delivered (received – this is what users call 'completed'), "
            "cancelled (cancelled or refunded). Users rarely use these words, "
            "so translate: completed/fulfilled/done -> delivered; "
            "refunded/void -> cancelled; in transit/dispatched -> shipped; "
            "unpaid -> pending. If nothing fits, ask the user."))

    created_on_or_after: date = Field(
        description="INCLUSIVE lower bound, an ISO 8601 date 'YYYY-MM-DD'. "
        "Orders created at any time on this date are included.")

    created_strictly_before: date = Field(
        description=(
            "EXCLUSIVE upper bound, an ISO 8601 date 'YYYY-MM-DD'. Orders "
            "created on this date are NOT included, so to cover a range that "
            "ends today you must pass TOMORROW's date. Example – today is "
            "2026-09-06 and the user says 'the last week': pass "
            "created_on_or_after='2026-08-30' and "
            "created_strictly_before='2026-09-07'."))

    max_results: int = Field(
        20, ge=1, le=500,
        description="Maximum records to return, 1 to 500. 'top 5' -> 5.")

    @field_validator("customer_account_id", mode="before")
    @classmethod
    def _not_a_company_name(cls, v):
        if isinstance(v, str) and v.strip() and not re.match(r"^CUST-\d{5}$", v.strip()):
            raise ValueError(
                f"'{v}' is not a valid account id (expected 'CUST-' followed by "
                f"five digits, e.g. 'CUST-40912'). If this is a company or "
                f"person name, resolve it with lookup_customer_id first, or "
                f"pass null to search all customers. Do not fabricate an id.")
        return v

    @field_validator("order_status", mode="before")
    @classmethod
    def _closed_vocabulary(cls, v):
        if isinstance(v, str) and v.strip().lower() not in (
            "pending", "paid", "shipped", "delivered", "cancelled"):
            raise ValueError(_closed_set_message(
                "order_status", v,
                ("pending", "paid", "shipped", "delivered", "cancelled"),
                STATUS_SYNONYMS))
        return v.strip().lower() if isinstance(v, str) else v

```

```

@field_validator("created_on_or_after", "created_strictly_before",
                 mode="before")

@classmethod
def _real_dates_only(cls, v):
    if isinstance(v, str) and not re.match(r"^\d{4}-\d{2}-\d{2}$", v.strip()):
        raise ValueError(
            f"'{v}' is not an ISO 8601 date. Use YYYY-MM-DD, e.g. "
            f"'2026-09-01'. Resolve relative phrases such as 'last week' "
            f"to a concrete date before calling.")
    return v

@model_validator(mode="after")
def _window_must_contain_a_day(self):
    if self.created_strictly_before <= self.created_on_or_after:
        raise ValueError(
            f"created_strictly_before ({self.created_strictly_before}) must "
            f"be strictly after created_on_or_after "
            f"({self.created_on_or_after}); as given, the window covers no "
            f"days. The upper bound is exclusive: to include today, pass "
            f"tomorrow's date.")
    return self

def search_orders(**kwargs):
    """Search the order database. See SearchOrdersArgs for the parameters."""
    a = SearchOrdersArgs(**kwargs)
    return {"query": a.model_dump(mode="json"), "results": []}

# -----
# Tool 3 – convert_currency
# -----
class ConvertCurrencyArgs(StrictModel):
    """Arguments for convert_currency.

    The two currency fields are named after WHAT THEY DESCRIBE, not after a
    direction, because 'from'/'to' invert easily when a sentence names the
    destination first.
    """

    amount: float = Field(
        gt=0,
        description="A positive number, already stripped of symbols and "
        "separators. '$1,200' -> 1200.0. Never pass a string.")

    amount_currency_code: CurrencyCode = Field(
        description=(
            "The currency `amount` is written in, as an uppercase ISO 4217 "
            "code. Ask yourself: what unit is the NUMBER in? That is this "
            "field, regardless of word order. In \"What's the euro price of an "
            "$89 subscription?\" the number is 89 dollars, so this is 'USD' – "
            "'euro' appears first but describes the ANSWER.))")

    result_currency_code: CurrencyCode = Field(
        description="The currency the answer comes back in, as an uppercase "
        "ISO 4217 code. Must differ from amount_currency_code.")

    rate_date: Optional[date] = Field(
        None,
        description="ISO 8601 date 'YYYY-MM-DD' for a historical rate, or null "
        "for the latest. Resolve phrases like 'the first of "
        "August' to a concrete date before calling.")

    @field_validator("amount", mode="before")
    @classmethod
    def _a_number_not_text(cls, v):
        if isinstance(v, str):
            raise ValueError(
                f"expected a number, got the string '{v}'. Strip symbols and "
                f"separators first: '$1,200' -> 1200.0.")

```

```

        return v

    @field_validator("amount_currency_code", "result_currency_code",
                     mode="before")
    @classmethod
    def _iso_codes_only(cls, v):
        if not isinstance(v, str):
            return v
        norm = v.strip().upper()
        if norm in FX:
            return norm
        hint = ""
        if norm in CURRENCY_NAMES:
            hint = (f" '{v}' is a currency name or symbol, not a code – use "
                    f" '{CURRENCY_NAMES[norm]}'.")
        raise ValueError(
            f" '{v}' is not a supported ISO 4217 currency code. Supported: "
            f" {', '.join(sorted(FX))}. {hint}")

    @model_validator(mode="after")
    def _two_different_currencies(self):
        if self.amount_currency_code == self.result_currency_code:
            raise ValueError(
                f"amount_currency_code and result_currency_code are both "
                f" '{self.amount_currency_code}', so there is nothing to "
                f"convert. Re-read the sentence: the currency the NUMBER is "
                f"written in goes in amount_currency_code; the currency of the "
                f"ANSWER goes in result_currency_code.")
        return self

def convert_currency(**kwargs):
    """Convert a money amount between currencies.

    NOTE: Pydantic cannot catch a swap of two valid codes. If the model puts
    'EUR' where 'USD' belongs, both values are legal members of the Literal
    and the call succeeds with the wrong answer. Only the field descriptions
    above prevent that.
    """
    a = ConvertCurrencyArgs(**kwargs)
    rate = FX[a.result_currency_code] / FX[a.amount_currency_code]
    return {"amount": round(a.amount * rate, 2),
            "currency": a.result_currency_code, "rate": round(rate, 6),
            "as_of": a.rate_date.isoformat() if a.rate_date else "latest"}

# -----
# Tool 4 – send_notification
# -----
ADDRESS_SPEC = {
    "email": (RE_EMAIL, "an email address of the form name@domain.tld",
              "marco.silva@example.com"),
    "sms": (RE_E164, "an E.164 phone number starting with '+',
            "+14155550142"),
    "slack": (RE_SLACK, "a Slack handle '@name' or member id 'U04AB12CD'",
              "@dana"),
    "push": (RE_PUSH, "a device token of at least 32 characters",
             "d9f1c0a4e77b4e2b9a3c5d6e7f8091a2"),
}

class SendNotificationArgs(StrictModel):
    """Arguments for send_notification."""

    channel: Channel = Field(
        description="CLOSED SET – exactly one of: email, sms, slack, push. "
        "Map the user's verb: 'text' -> sms, 'DM'/'ping on Slack' "
        "-> slack, 'notify on their phone' -> push.")

    recipient_address: str = Field(
        description=

```

```

        "The address in the format REQUIRED BY `channel` – the formats are "
        "not interchangeable. email -> name@domain.tld; sms -> an E.164 "
        "number like '+14155550142'; slack -> '@handle' or a member id "
        "like 'U04AB12CD'; push -> a 32+ character device token. You will "
        "often hold only someone's email. That does NOT make it usable on "
        "another channel: call lookup_contact(person, channel) instead.")

message_body: str = Field(
    min_length=1, max_length=1000,
    description="Plain text, 1 to 1000 characters. No markup, no "
    "attachments.")

priority: Priority = Field(
    "normal",
    description="CLOSED SET – exactly one of: low, normal, high. Map the "
    "user's wording: urgent/ASAP/critical/P0 -> high; "
    "medium/standard -> normal; FYI/no rush -> low. There is "
    "no level above 'high'.")

@field_validator("channel", mode="before")
@classmethod
def _known_channel(cls, v):
    if isinstance(v, str) and v.strip().lower() not in (
        "email", "sms", "slack", "push"):
        raise ValueError(_closed_set_message(
            "channel", v, ("email", "sms", "slack", "push"),
            CHANNEL_SYNONYMS))
    return v.strip().lower() if isinstance(v, str) else v

@field_validator("priority", mode="before")
@classmethod
def _known_priority(cls, v):
    if isinstance(v, str) and v.strip().lower() not in ("low", "normal", "high"):
        raise ValueError(_closed_set_message(
            "priority", v, ("low", "normal", "high"), PRIORITY_SYNONYMS))
    return v.strip().lower() if isinstance(v, str) else v

@model_validator(mode="after")
def _address_must_match_channel(self):
    pattern, human, example = ADDRESS_SPEC[self.channel]
    if not pattern.match(self.recipient_address):
        extra = ""
        if RE_EMAIL.match(self.recipient_address) and self.channel != "email":
            extra = (f" You passed an email address with "
                    f"channel='{self.channel}'. Call "
                    f"lookup_contact(person, '{self.channel}') to get "
                    f"their {self.channel} address, or send it by email.")
        raise ValueError(
            f"recipient_address '{self.recipient_address}' is not valid for "
            f"channel='{self.channel}', which requires {human}. Example: "
            f"'{example}'.{extra}")
    return self

def send_notification(**kwargs):
    """Send one notification to one person. See SendNotificationArgs."""
    a = SendNotificationArgs(**kwargs)
    return {"message_id": "msg_0001", "channel": a.channel,
            "to": a.recipient_address, "priority": a.priority,
            "chars": len(a.message_body)}

# -----
# Tool 5 – resize_image
# -----

class ResizeImageArgs(StrictModel):
    """Arguments for resize_image.

    This tool understands ABSOLUTE PIXELS ONLY. It cannot scale by a
    percentage and it does not know the source image's current size. For a
    relative request ('half the size', '25% smaller'), call

```

```

get_image_info(source_path) first, do the arithmetic, and pass real
pixels. Never send 50 to mean '50%' or 0.5 to mean 'half'.
"""

source_path: str = Field(min_length=1, description="Path of the image to read.")
target_path: str = Field(min_length=1, description="Path to write to.")

width_px: Optional[int] = Field(
    None, ge=1, le=20000,
    description="Target width in WHOLE PIXELS, or null to derive it from "
        "height_px. 'resize to 800 pixels wide' -> width_px=800, "
        "height_px=null. Do NOT repeat the width as the height.")

height_px: Optional[int] = Field(
    None, ge=1, le=20000,
    description="Target height in WHOLE PIXELS, or null to derive it from "
        "width_px. At least one of width_px / height_px must be "
        "an integer.")

preserve_aspect_ratio: bool = Field(
    True,
    description="True (default) fits the image inside the given box "
        "without distortion. False stretches it to exactly "
        "width_px x height_px and requires BOTH. The word "
        "'exactly' in a request means False.")

jpeg_quality: int = Field(
    85, ge=1, le=100,
    description="Encoder quality 1-100. Only used for .jpg/.jpeg targets. "
        "'high quality' -> 95.")

@field_validator("width_px", "height_px", mode="before")
@classmethod
def _pixels_not_percentages(cls, v):
    if isinstance(v, float) and 0 < v < 1:
        raise ValueError(
            f"{v} looks like a scale factor, not a pixel count. This tool "
            f"takes absolute pixels only – call "
            f"get_image_info(source_path), multiply the real dimension "
            f"yourself, and pass the result, e.g. 960.")
    if isinstance(v, str):
        raise ValueError(
            f"'{v}' is not a number. Pass a whole number of pixels, e.g. "
            f"800 – not '800px' and not a percentage.")
    return v

@model_validator(mode="after")
def _dimensions_make_sense(self):
    if self.width_px is None and self.height_px is None:
        raise ValueError(
            "at least one of width_px / height_px must be given. Use "
            "width_px for 'N pixels wide', height_px for 'N pixels tall', "
            "or both for a bounding box. For a relative request such as "
            "'half the size', read the current size with "
            "get_image_info(source_path) and compute the pixels first.")
    if not self.preserve_aspect_ratio and (
        self.width_px is None or self.height_px is None):
        raise ValueError(
            "an exact (non-proportional) resize needs both width_px and "
            "height_px. Either supply the missing one, or set "
            "preserve_aspect_ratio=True to scale from the one you have.")
    return self

def resize_image(**kwargs):
    """Resize an image file. See ResizeImageArgs."""
    a = ResizeImageArgs(**kwargs)
    return {"path": a.target_path, "width": a.width_px, "height": a.height_px,
        "quality": a.jpeg_quality, "fit": a.preserve_aspect_ratio}

```

```

TOOLS = {"schedule_meeting": schedule_meeting,
         "search_orders": search_orders,
         "convert_currency": convert_currency,
         "send_notification": send_notification,
         "resize_image": resize_image}

ARG_MODELS = {"schedule_meeting": ScheduleMeetingArgs,
              "search_orders": SearchOrdersArgs,
              "convert_currency": ConvertCurrencyArgs,
              "send_notification": SendNotificationArgs,
              "resize_image": ResizeImageArgs}

def tool_schema(name: str) -> dict:
    """The JSON Schema an LLM framework sends as the tool definition."""
    return {"name": name,
            "description": (TOOLS[name].__doc__ or "").strip().split("\n")[0],
            "input_schema": ARG_MODELS[name].model_json_schema()}

```

Appendix B — how to run this yourself

Unzip the files into a folder and run:

```
pip install pydantic      # the only dependency, and only for round 3

python3 run_round1.py     # the original tools: 6 right, 9 wrong
python3 run_round2.py     # fixed, hand-written checks: 15 right, 18 caught
python3 run_round3.py     # fixed, Pydantic checks:      15 right, 18 caught
```

What each file is for:

FILE	WHAT IT HOLDS
<code>tools_v1.py</code>	The five original tools. Good names, detailed docstrings, no input checks.
<code>tools_v2.py</code>	The same five, rewritten. New docstrings, with the checks hand-written as <code>if</code> statements.
<code>tools_v3.py</code>	The version in Appendix A: the same rules declared in Pydantic models, which also generate the JSON Schema the model reads.
<code>cases.py</code>	The 15 requests, the arguments the model sent, and the arguments a correct answer needed.
<code>cases_v2.py</code>	The retest, plus the 18 deliberately broken calls used to test the error messages.
<code>run_round1.py</code>	Runs the first round and writes <code>round1_results.json</code> .
<code>run_round2.py</code>	Runs the retest and the check suite against the hand-written version.
<code>run_round3.py</code>	The same against the Pydantic version, and prints the generated JSON Schema.

To try it with your own tools, replace the five in `tools_v1.py`, write three natural phrasings per tool in `cases.py`, and record what your model sends back.